



S Y N A R T E S I S

An undo layer
for AI agents

PALANTIR STARTUP FELLOWSHIP · COHORT 003

ARHAAN KHAN · PHAGWARA, PUNJAB, INDIA

THE PROBLEM

AN AGENT RUNS TWENTY STEPS AND GETS STEP SEVEN WRONG

It has write access to a real system. It misreads one step and applies the remaining thirteen to the wrong records. You find out an hour later.

WHAT YOU CAN DO TODAY

THREE BAD OPTIONS

- **Reverse it by hand** from the transcript, if there is one, and if you can tell which of the forty calls were the wrong ones.
- **Restore a backup** and lose every legitimate change anyone made in the same window.
- **Accept the damage.**

Sandboxes do not help: the container is disposable, the CRM row it updated over the network is not. Tracing does not help: a log says **update_customer** ran forty times, not what the values were before.

WHAT IT IS

A PROXY THAT REMEMBERS WHAT EVERY CALL REPLACED

Synartesis sits between an AI client and the servers it calls. Every tool call is recorded together with **the state that call replaced**, and that state can be put back: one run, walked backwards, newest call first.

What cannot be put back, it refuses to let an agent do unsupervised.

The agent sees the same tools, the same names, the same results. Nothing about how you work changes.

THE MODEL

EVERY TOOL GETS ONE OF FOUR CLASSIFICATIONS

READONLY

Changes nothing. Recorded, forwarded.

REVERSIBLE

Prior state read before the write. Written back on undo.

COMPENSABLE

Cannot be reversed. A second call neutralises it.

IRREVERSIBLE

Neither. Held until a person approves it.

A tool the policy does not name is treated as **irreversible** and held. Silently forwarding an unknown destructive call is the one failure worth avoiding most.

PROOF

101

FILES WRECKED IN 0.2 SECONDS

1.15s

TO PUT EVERY ONE OF THEM BACK

A real agent, through the proxy, against a real 483-file open-source repository. 284 lines changed across 101 files.

synartesis undo restored all of them. **git status** came back clean.

Six concurrent agents in the same journal: 180 writes, zero errors, integrity check ok.

THE PART THAT MATTERS

IT REFUSES

Anything can write an old value back. Declining to is the hard part.

Before undoing a step, Synartesis re-reads the resource and compares it against the state that step left behind. If somebody has been there since, **it stops** and shows the lines that differ.

Undo walks newest first, so whatever it had already put back stays put back.

```
halted at sequence 1  drift detected
```

```
the resource is not in the  
state this run left it in.
```

```
at line 1:
```

```
- AGENT VERSION
```

```
+ A HUMAN FIXED THIS BY HAND
```

```
1 removed, 1 added.
```

```
R E S U L T  partial
```

STATUS

SHIPPED, NOT A PROTOTYPE

ON NPM

v0.3.2, ten releases since 22 August. 1,118 downloads in the last 30 days.

TESTED

300 tests across macOS and Linux, Node 22 and 24. Every release verified against the registry before tagging.

OPEN

MIT. ~9,900 lines of TypeScript, 130 commits. Four finished policies: filesystem, git, GitHub, memory.

The journal is a secrets-bearing store, so it is created **0600** in a **0700** directory and re-tightened on every open. **synartesis prune** is how history stops accumulating. Nothing still active or waiting on a person is ever pruned.

WHY FOUNDRY

THE ONTOLOGY IS THE BEST SUBSTRATE THIS COULD HAVE

Today Synartesis wraps servers whose write semantics it has to **infer from a tool schema**. Foundry is the opposite: Action types are declared, typed and versioned.

That makes classification and inverse-resolution far more reliable than anything possible over generic MCP. And AIP agents already write to the Ontology, which is exactly where an agent's mistake is most expensive.

This is not a port. It is the version of the product with the right substrate underneath it.

WHAT WE WOULD BUILD

SYNARTESIS FOR THE ONTOLOGY

- **Ontology & Action Types** — Actions are the write primitive, so classification and inverse-resolution attach at the Action type level.
- **OSDK** — the read-before-write that captures prior object state, and the re-read at rollback that detects a human edit since.
- **AIP Agent Studio / Logic** — where the agent lives. The proxy sits between it and its Action tools.
- **Foundry Functions** — server-side resolution of compensating Actions where reversal is impossible but offsetting is not.
- **Datasets** — the journal lands as one, so what an agent did and what it replaced is queryable alongside everything else.
- **Manual review on Actions** — the approval gate.

TEN WEEKS

MILESTONES

25% · LATE OCT

Foundry-native policy reading Action type metadata. One Action type round-tripped: agent invokes, state captured via OSDK, undo restores. Journal landing as a dataset.

50% · BUILD CAMP

All four classes working against Ontology Actions, including drift detection. An approval gate on a real irreversible Action type. Demoable on a live workflow, not a fixture.

100% · EARLY DEC

An AIP agent making a run of Ontology writes, one held for approval, the whole run reversed on one command with a human's edits intact.

WHO

ARHAAN KHAN

Self-taught. Solo founder. Phagwara, Punjab, India. Available on IST, able to travel self-funded for the in-person events.

Synartesis is not incorporated and has raised nothing. It exists because I wanted to give an agent write access to something I cared about and could not find a way back.

`synartesis.online`
`github.com/ArhaanDev24/Synartesis`
`npm install -g synartesis`
`linkedin.com/in/arhaankhan143`